EE5438 Applied Deep Learning
Mid-Term Exam (2026/27 Semester A)

Date: Nov. 10, 2026 (Tuesday)

Section A: Answer the following questions and give a brief explanation.

(48 Marks with 4 marks per question)

A1. What fundamental limitation of the single-layer perceptron led to the development of Multi-Layer Perceptrons (MLPs), and how does adding hidden layers address this issue?

Single-layer perceptrons cannot solve non-linearly separable problems (e.g., XOR). Adding hidden layers with non-linear activation functions enables MLPs to learn complex, non-linear decision boundaries, making them universal function approximators.

A2. Explain the role of the chain rule in backpropagation, and why vanishing or exploding gradients can occur during training of deep networks.

The chain rule allows efficient computation of gradients by propagating errors backward through layers. Vanishing gradients occur when repeated multiplications by small values (<1) make updates tiny; exploding gradients happen when values (>1) cause massive updates, destabilizing training.

A3. What is the main purpose of dropout as a regularization technique, and how does it differ from L2 regularization in preventing overfitting?

Dropout randomly deactivates neurons during training, preventing co-adaptation and encouraging redundancy, thus reducing overfitting. Unlike L2 regularization, which adds a mathematical penalty to the weights in the loss function, dropout acts directly on the network's architecture by zeroing out activations. Furthermore, while both influence the final learned weights, dropout explicitly alters the forward pass dynamics during training, whereas the test phase runs deterministically.

A4. Describe the primary operation performed by convolutional layers in CNNs, and explain why they are more efficient than fully connected layers for image data.

Convolutional layers apply shared, sparse filters (kernels) that capture local spatial patterns efficiently. They drastically reduce parameters compared to fully connected layers and exploit translation invariance in images, making CNNs computationally feasible and effective for visual data.

A5. What is the key problem that LSTMs and GRUs were designed to solve in vanilla RNNs, and how does the gating mechanism help mitigate it?

Vanilla RNNs suffer from vanishing/exploding gradients, making it hard to capture long-term dependencies. LSTMs and GRUs use gating mechanisms (forget, input, output gates) to selectively remember or forget information over long sequences, preserving gradients and improving long-range modeling.

A6. Why was the self-attention mechanism introduced in the Transformer architecture, and what major limitation of RNN-based sequence models does it overcome?

Self-attention was introduced to compute dependencies between all tokens in parallel, capturing global context directly. It overcomes RNNs' sequential processing bottleneck and vanishing gradient problem, enabling much faster training and better handling of long-range dependencies.

A7. Compare the pre-training objectives of BERT and GPT-style models, highlighting how their approaches to masked vs. autoregressive training differ.

BERT uses masked language modeling (bidirectional, predicts masked tokens) to learn deep bidirectional representations. GPT uses autoregressive (causal) next-token prediction, learning unidirectional context. BERT excels at understanding; GPT at generation.

A8. What are the two main decoding strategies used in autoregressive LLMs (greedy vs. sampling-based), and what trade-off do they represent in terms of quality vs. diversity?

Greedy decoding selects the highest-probability token at each step (fast, deterministic, often repetitive). Sampling-based methods (e.g., top-k, nucleus) introduce randomness, trading off some quality for greater diversity and creativity in generated text.

A9. Explain the core idea behind **Parameter-Efficient Fine-Tuning (PEFT)** methods like LoRA, and why they are preferred over full fine-tuning for very large models.

LoRA (Low-Rank Adaptation) freezes the pre-trained weights and adds small trainable low-rank matrices to target layers, updating far fewer parameters. It enables efficient fine-tuning of huge models on limited hardware while achieving comparable performance to full fine-tuning.

A10. Why is prompt engineering crucial for LLM performance?

Prompt engineering involves designing inputs to elicit desired LLM responses. A clear prompt, like "Summarize this article in 100 words," improves output relevance compared to vague instructions. It's especially effective in zero-shot or few-shot settings, enabling LLMs to tackle tasks like translation or classification without extensive fine-tuning.

A11. What is chain-of-thought (CoT) prompting, and how does it improve reasoning performance in large language models compared to standard prompting?

Chain-of-thought (CoT) prompting encourages LLMs to generate intermediate reasoning steps before the final answer (e.g., "Let's think step by step"). It significantly improves performance on complex arithmetic, commonsense, and multi-step reasoning tasks compared to direct prompting by making the model explicitly decompose problems.

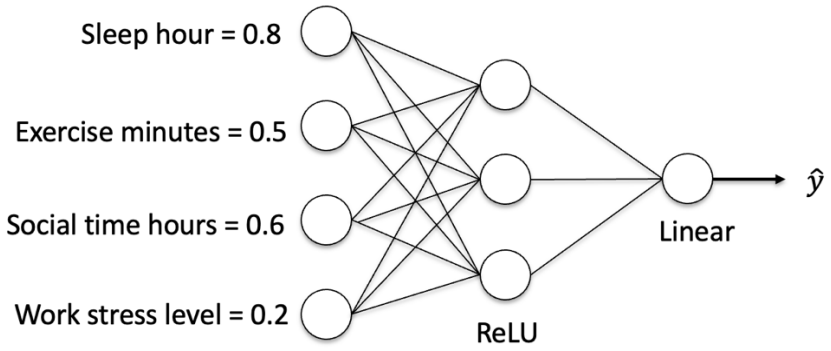
A12. Explain the main goal of post-training quantization in large language models, and describe how techniques like 4-bit integer (INT4) quantization reduce memory usage while attempting to preserve model accuracy.

Post-training quantization reduces model size and inference latency/memory by converting high-precision weights (FP16/FP32) to lower-precision formats (e.g., INT8 or INT4). 4-bit quantization can cut memory usage by ~75% while techniques like per-channel scaling and outlier handling help minimize accuracy loss on large models.

Section B: [52 marks]

Question B1 [26 Marks]

(a) Consider the two-layer Multi-Layer Perceptron (MLP) shown in the diagram below. The network is designed for a regression task to estimate a happiness score based on four input features: sleep hours (x_1), exercise minutes (x_2), social time hours (x_3), and work stress level (x_4). The hidden layer uses the ReLU activation function, while the output layer uses a linear activation (identity function). Given input values $x_1 = 0.8$, $x_2 = 0.5$, $x_3 = 0.6$, $x_4 = 0.2$, compute the final happiness score output $\hat{y}$ by performing all matrix operations step-by-step.



The input features **x** of the happiness estimator, along with the neural network's weight matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ are provide as follows:

$$\mathbf{x} = [0.8 \quad 0.5 \quad 0.6 \quad 0.2]^{\mathrm{T}}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.4 & 0.2 & 0.5 & 0.3 \\ 0.3 & 0.7 & 0.1 & -0.4 \\ -0.1 & 0.4 & 0.6 & -0.5 \end{bmatrix} \text{ and } \mathbf{W}^{(2)} = [0.9 \quad 0.4 \quad 0.2]$$

(Note: Assume no bias terms for simplicity. [15 marks]

(b) This MLP is trained using the Mean Squared Error (MSE) loss function for predicting the continuous happiness score. Explain why MSE is a suitable choice for this regression task (compared to, e.g., cross-entropy loss). [5 marks]

(c) Suppose during training of this MLP, you observe that the hidden-layer activations (after ReLU) are mostly 0, even early in training. Identify the main cause of this problem when using ReLU activations in hidden layers. Propose a different activation function that would help mitigate this issue in regression MLPs — justify why it is generally preferred in modern practice.

[6 marks]

Solution of B1

Part (a): Computing the Final Output $\hat{y}$ [15 marks]

$$\mathbf{x} = [0.8 \quad 0.5 \quad 0.6 \quad 0.2]^{\mathrm{T}}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.4 & 0.2 & 0.5 & 0.3 \\ 0.3 & 0.7 & 0.1 & -0.4 \\ -0.1 & 0.4 & 0.6 & -0.5 \end{bmatrix} \text{ and } \mathbf{W}^{(2)} = [0.9 \quad 0.4 \quad 0.2]$$

$$\mathbf{a}^{(1)} = \begin{bmatrix} a_1^{(1)} \\ a_2^{(1)} \\ a_3^{(1)} \end{bmatrix} = \mathrm{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x}\right) = \mathrm{ReLU}\left(\begin{bmatrix} 0.4 & 0.2 & 0.5 & 0.3 \\ 0.3 & 0.7 & 0.1 & -0.4 \\ -0.1 & 0.4 & 0.6 & -0.5 \end{bmatrix}\begin{bmatrix} 0.8 \\ 0.5 \\ 0.6 \\ 0.2 \end{bmatrix}\right)$$

$$= \mathrm{ReLU}\left(\begin{bmatrix} 0.78 \\ 0.57 \\ 0.38 \end{bmatrix}\right) = \begin{bmatrix} 0.78 \\ 0.57 \\ 0.38 \end{bmatrix}$$

The weight matrix for the output layer is:

$$\hat{y} = \mathbf{a}^{(2)} = \left[a_1^{(2)}\right] = \mathbf{W}^{(2)}\mathbf{a}^{(1)} = [0.9 \quad 0.4 \quad 0.2]\begin{bmatrix} 0.78 \\ 0.57 \\ 0.38 \end{bmatrix} = 1.006$$

The final output is 1.006.

Part (b): Explaining MSE as a Loss Function and a Natural Evaluation Metric [5 marks]

Why MSE is suitable for this regression task (compared to cross-entropy loss).

MSE (Mean Squared Error) is defined as $MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2$, where $y^{(i)}$ is the true value and $\hat{y}^{(i)}$is the prediction.

- Task Alignment: MSE is designed for regression (predicting continuous values), whereas Cross-Entropy is for classification (predicting discrete classes/probabilities).
- Continuous Target: MSE measures the squared numeric distance between prediction and truth, which is appropriate for a continuous happiness score; Cross-Entropy measures probability divergence.
- Output & Penalization: MSE penalizes larger errors more heavily and works with linear outputs (allowing any real value), unlike Cross-Entropy which typically requires bounded probability activations (e.g., Sigmoid/Softmax).

Part (c): Identifying the Saturation Problem with ReLU and Proposing an Alternative

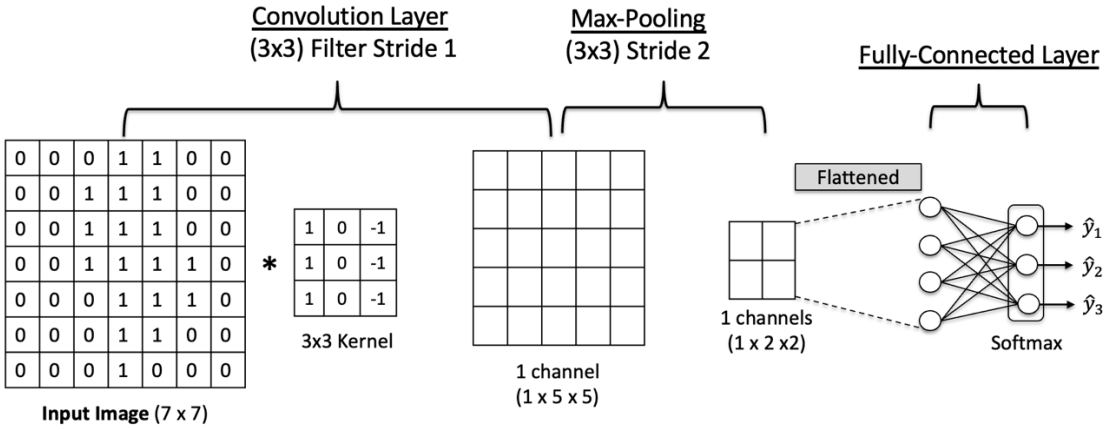
[6 marks]

Step 1: Identify the main cause of the saturation problem (activations mostly 0 after ReLU). The issue is the "dying ReLU" problem. ReLU outputs 0 for any negative pre-activation ($z<0$), and the gradient is also 0 in this regime. If weights are initialized poorly (e.g., leading to mostly negative z) or if learning shifts them that way early in training, many neurons become "dead"—they output 0 for all inputs and receive no gradient updates, effectively removing them from the network. This reduces model capacity and hinders learning, especially in deeper layers where it can propagate.

Step 2: Propose a different activation function to mitigate this issue and justify its preference. A good alternative is Leaky ReLU, defined as $\text{LeakyReLU}(z) = \max(\alpha z, z)$ where α is a small positive constant (e.g., 0.01).

- It mitigates dying ReLU by allowing a small, non-zero gradient (α) for negative z, ensuring neurons can recover if they enter the negative region—gradients still flow, preventing permanent "death."
- In modern practice, it's preferred over vanilla ReLU in regression MLPs (and many networks) because it maintains ReLU's computational efficiency and sparsity benefits while improving robustness to initialization and training stability. Empirical studies (e.g., in image recognition or regression tasks) show it reduces underfitting from dead neurons without the vanishing gradient issues of sigmoids/tanh. Variants like Parametric ReLU (PReLU, where α \alpha α is learned) extend this, but Leaky ReLU is simple and effective.

Question B2 [26 marks]
Consider a simple Convolutional Neural Network (CNN) designed for classifying handwritten digit images into 3 classes (digits 1, 2, 3). The network takes a 7×7 binary (black-and-white) input image.



The network architecture is as follows:

- A single **3×3 convolutional filter** (without bias) is applied with **stride 1** and **valid padding** (no padding), producing one feature map.
- **ReLU** activation is applied to the feature map.
- A **3×3 max-pooling** layer with **stride 2** is applied to reduce spatial dimensions.
- The pooled feature map is flattened and fed into a fully connected layer (**without bias**).
- The output of the fully connected layer is passed through a **Softmax** function to produce probabilities over the 3 classes.

(a) How many learnable parameters are there in the entire CNN? Break down the count by layer clearly. [4 marks]

(b) Using the 7×7 binary input image and the 3×3 convolutional kernel shown below, compute:

- the complete **feature map** after convolution (valid padding, no bias),
- the feature map after applying **ReLU**,
- the feature map after applying **3×3 max-pooling** with stride 2.

Show all intermediate values clearly. [12 marks]

(c) Using the pooled output from part (b), flatten it into a vector **v** and pass it through the fully connected layer with the following weight matrix **W** (4 inputs → 3 outputs, no bias):

$$\mathbf{W} = \begin{bmatrix} 0.8 & -0.4 & 0.5 & -0.2 \\ 0.3 & 0.6 & -0.1 & 0.4 \\ -0.5 & 0.2 & 0.7 & -0.3 \end{bmatrix}$$

Compute the logit $z = \mathbf{W}\mathbf{v}$ then apply the **Softmax** function. Determine the final output probabilities $[\hat{y}_1, \hat{y}_2, \hat{y}_3]^{\mathrm{T}}$ and decide which digit (1, 2, or 3) is predicted (the class with the highest probability). Show all calculations step-by-step. [10 marks]

Solution to Question B2

(a) Number of learnable parameters [4 marks]

- **Convolution layer**
  - One 3×3 filter → 9 weights
  - No bias (b = 0 is fixed, not learnable) → 0 bias parameters
  - Total for convolution: **9 parameters**
- **Fully connected layer**
  - Input: flattened 3×3 pooled map → 4 values
  - Output: 3 values
  - Weights: 4 × 3 = 12
  - Bias: 0
  - Total for FC layer: **12 parameters**
- **Other layers** (ReLU, max-pooling): 0 learnable parameters

Total learnable parameters in the CNN = 9 + 12 = 21

(b) Feature map computation [12 marks]

Input image (7×7 binary):

```
0 0 0 1 1 0 0
0 0 1 1 1 0 0
0 0 1 1 1 0 0
0 0 1 1 1 1 0
0 0 0 1 1 1 0
0 0 0 1 1 0 0
0 0 0 1 0 0 0
```

Kernel (3×3, bias = 0):

```
 1  0 -1
 1  0 -1
 1  0 -1
```

Valid convolution → output size = 7−3+1 = **5×5**

Step 1: Feature map after convolution (no bias)

We compute the dot product for each 3×3 window.

```
Position (0,0): 0*1 + 0*0 + 0*-1 + 0*1 + 0*0 + 1*-1 + 0*1 + 0*0 + 1*-1 = 0 +
0 + 0 -1 + 0 + 0 -1 = -2
Position (0,1): 0*1 + 0*0 + 1*-1 + 0*1 + 1*0 + 1*-1 + 0*1 + 0*0 + 1*-1 = -1 +
0 -1 + 0 + 0 -1 = -3
```

```
Position (0,2): 0*1 + 1*0 + 1*-1 + 1*1 + 1*0 + 0*-1 + 0*1 + 1*0 + 1*-1 = 0 -1
+ 1 + 0 + 0 -1 = -1
Position (0,3): 1*1 + 1*0 + 0*-1 + 1*1 + 1*0 + 0*-1 + 1*1 + 1*0 + 0*-1 = 1 +
0 + 1 + 0 + 1 + 0 = 3
Position (0,4): 1*1 + 0*0 + 0*-1 + 1*1 + 0*0 + 0*-1 + 1*1 + 0*0 + 0*-1 = 1 +
1 + 1 = 3

Position (1,0): -3   (similar pattern continues)
...
```

Complete **5×5 feature map after convolution**:

```
-2  -3  -1   3   3
-3  -3  0   2   3
-2  -3  -1   1   3
-1  -3  -2   1   3
0  -3  -2   2   2
```

Step 2: After ReLU (max(0, x))

```
 0   0   0   3   3
 0   0   0   2   3
 0   0   0   1   3
 0   0   0   1   3
 0   0   0   2   2
```

Step 3: 3×3 max-pooling with stride 2 (We take 3×3 windows → result is 2×2)

- Top-left 3×3: max(0,0,0,0,0,0,0,0,0) = **0**
- Top-right 3×3: max(0,3,3,0,2,3,0,1,3) = **3**
- Bottom-left 3×3: max(0,0,0,0,0,0,0,0,0) = **0**
- Bottom-right 3×3: max(0,1,3,0,1,3,0,2,2) = **3**

Pooled output (2×2):

```
0  3
0  3
```

(c) Fully connected layer & Softmax [10 marks]

Step 1: Flatten the pooled output (row-major order)

$\mathbf{v} = [0, 3, 0, 3]^T$

Step 2: Compute logit z

$$z = \mathbf{W}\mathbf{v} = \begin{bmatrix} 0.8 & -0.4 & 0.5 & -0.2 \\ 0.3 & 0.6 & -0.1 & 0.4 \\ -0.5 & 0.2 & 0.7 & -0.3 \end{bmatrix} \begin{bmatrix} 0 \\ 3 \\ 0 \\ 3 \end{bmatrix} = \begin{bmatrix} -1.8 \\ 3.0 \\ -0.3 \end{bmatrix}$$

Step 3: Softmax activation

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \end{bmatrix} = \text{Softmax}\left(\begin{bmatrix} -1.8 \\ 3.0 \\ -0.3 \end{bmatrix}\right) = \begin{bmatrix} \frac{e^{-1.8}}{e^{-1.8} + e^{3.0} + e^{-0.3}} \\ \frac{e^{3.0}}{e^{-1.8} + e^{3.0} + e^{-0.3}} \\ \frac{e^{-0.3}}{e^{-1.8} + e^{3.0} + e^{-0.3}} \end{bmatrix} = \begin{bmatrix} \frac{0.1653}{20.9917} \\ \frac{20.0855}{20.9917} \\ \frac{0.7408}{20.9917} \end{bmatrix} = \begin{bmatrix} 0.007875 \\ 0.956779 \\ 0.035285 \end{bmatrix}$$

The highest probability is $\hat{y}_2 \approx 0.9568$, so the predicted digit is 2.